CS341 CacheLab Design

Tomas Salaz

5 November 2025

Design

Intro

For the Design of the Cache Lab, I am going to implement an Array of Sets to simulate cache.
The Array is used for the Random Accesses to the elements, so we can quickly access the data in the cache.
The Sets become more complex because there isn't a data type. Since this is the case I will make custom structures that imitate how cache behaves.

Structure Details
Since we are implementing Least-Recently Used (LRU) eviction and this happens on a line by line basis we have to implement a `Line` struct. `Line` will contain `unsigned tag; unsigned LRU; unsigned valid;` fields to implement all the elements of a cache line that can contain multiple blocks.
Another struct is going to be *set* which will contain the lines with contiguous behavior. This struct will only contain `line *lines`.
The last struct is the cache itself, called `Cache`. This struct will contain the fields `set *sets; int S, E, B, s, e, b; unsigned time;`.

Implementation Details and Reasoning
Firstly we code the structs and intialize the cache as a whole using something like this `cache -> sets = malloc(S * sizeof(set))`, then use a loop to and continuous allocation `calloc`, to allocate all the space for the cache simulation.
Then we need a way to decode to get the info in the cache. We only need 2 fields `unsigned setidx; unsigned tag` which can be decoded with masks and shifts.

In Short
To simulate the Cache, we are organizing a 2D structure with 2^s Sets, each set containing E lines in one data Block.
Each line stores a `valid, tag, and LRU` to implement the LRU behavior.
When accessing an address, the simulator uses the `set index bit` to pick a set `tag bit` to find a match, and replaces the LRU line if there is a miss (Section 4.4), so this will be the main focus of the program.